

Arfan Nabil Ramadhan Emerald

THE LAST ASTRONAUT

INPUT n (jumlah astronot)

INPUT k (nilai eliminasi awal)

CREATE array astronot of size n

FOR i FROM 0 TO n-1:

 astronot[i] = i + 1

END FOR

sisa_astronot = n

index_sekarang = 0

PRINT "Urutan eliminasi: "

WHILE sisa_astronot > 1:

 langkah = k - 1

 // Cari astronot ke-K yang masih aktif

 WHILE langkah > 0:

 index_sekarang = (index_sekarang + 1) MOD n

 IF astronot[index_sekarang] != 0:

 langkah = langkah - 1

 END IF

 END WHILE

 tereliminasi = astronot[index_sekarang]

 PRINT tereliminasi

```

// Tandai tereliminasi dengan 0
astronot[index_sekarang] = 0
sis_astronot = sis_astronot - 1

// Perbarui nilai K
IF tereliminasi MOD 2 == 0:
    k = k + 2
ELSE:
    k = k - 1
END IF

IF k < 2:
    k = 2
END IF

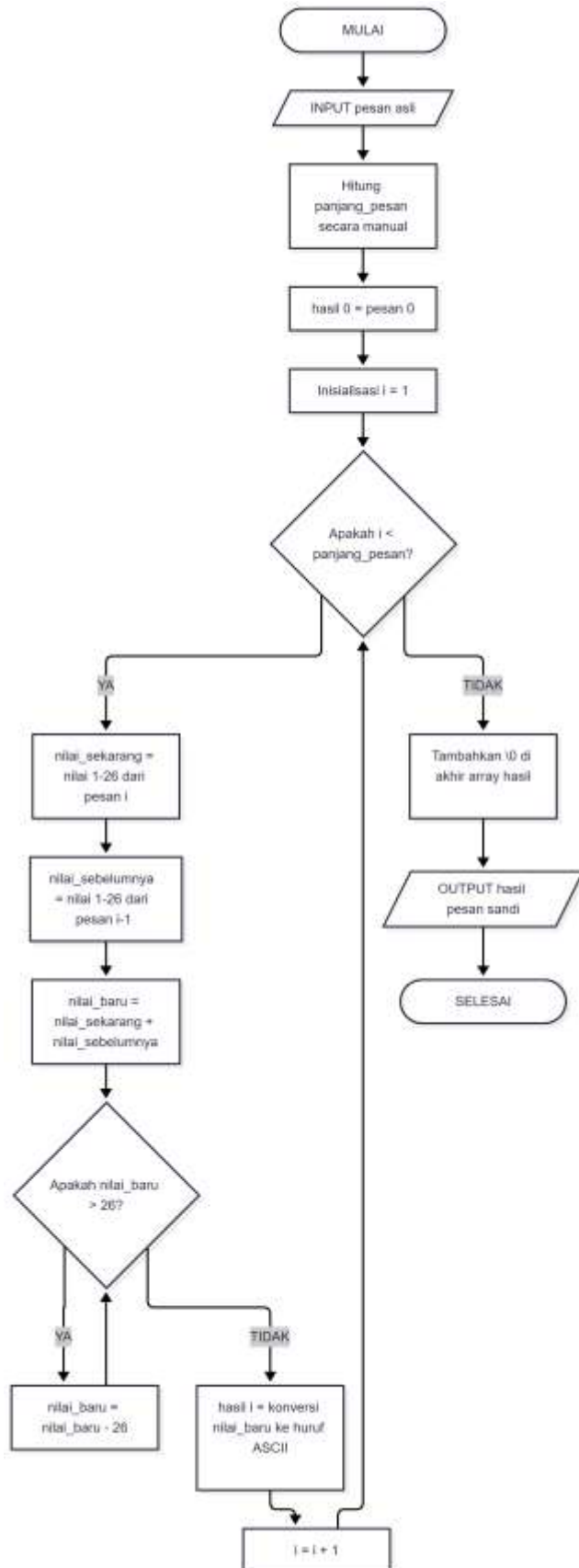
// Maju ke astronot berikutnya yang masih hidup
WHILE sis_astronot > 0 AND astronot[index_sekarang] == 0:
    index_sekarang = (index_sekarang + 1) MOD n
END WHILE
END WHILE

// Cari astronot terakhir
FOR i FROM 0 TO n-1:
    IF astronot[i] != 0:
        PRINT "Astronot terakhir: ", astronot[i]
        BREAK
    END IF
END FOR

```

Logika penyelesaian masalah "The Last Astronaut" menggunakan struktur data array untuk melacak posisi serta status seluruh astronot yang tersusun dalam lingkaran. Elemen array diinisialisasi dengan nomor unik astronot dari 1 hingga N. Untuk mensimulasikan mekanisme eliminasi melingkar tanpa perlu menghapus elemen dari memori secara fisik, algoritma memanfaatkan penanda nilai 0 bagi astronot yang telah tereliminasi dan menggunakan operasi *modulus* (MOD) untuk perputaran indeks. Proses penghitungan sebanyak K langkah dilakukan secara iteratif dengan melompati elemen yang bernilai 0 dan hanya menghitung astronot yang masih aktif. Ketika hitungan mencapai nilai K, astronot di posisi tersebut langsung dicetak ke urutan eliminasi dan nilainya diubah menjadi 0.

Setelah seorang astronot gugur, algoritma mengevaluasi nomor astronot tersebut untuk memperbarui nilai K secara dinamis. Apabila nomor astronot yang tereliminasi bernilai genap, nilai K bertambah 2; sebaliknya jika bernilai ganjil, nilai K berkurang 1. Algoritma juga menerapkan kondisi batas untuk memastikan nilai K tidak pernah kurang dari 2 dengan mereset nilainya menjadi 2 apabila $K < 2$. Posisi awal perhitungan berikutnya kemudian digeser ke astronot aktif pertama tepat setelah lokasi eliminasi. Seluruh siklus ini terus berjalan di dalam perulangan hingga variabel penanda sisa astronot bernilai 1, yang mengidentifikasi astronot terakhir yang berhasil bertahan.



Alien in the middle

Berdasarkan gambar flowchart yang diberikan, logika penyelesaian masalah penyandian ini diawali dengan menerima input pesan dan menghitung panjangnya secara manual. Sesuai dengan aturan sandi, huruf pertama dari pesan tidak mengalami perubahan sama sekali, sehingga nilai indeks pertama pada array hasil langsung diisi dengan karakter huruf pertama dari pesan asli (hasil 0 = pesan 0). Setelah langkah inisialisasi tersebut, program menggunakan variabel $i = 1$ untuk memulai proses perulangan yang akan memeriksa dan memodifikasi karakter kedua hingga karakter terakhir. Perulangan utama ini dirancang untuk terus berjalan selama nilai i masih lebih kecil dari total panjang pesan.

Di dalam perulangan tersebut, algoritma memproses pergeseran huruf dengan mengubah huruf pada indeks saat ini (pesan i) dan huruf tepat di urutan sebelumnya (pesan $i-1$) menjadi nilai angka berurutan dari 1 hingga 26. Angka-angka ini kemudian dijumlahkan untuk membentuk sebuah nilai_baru. Karena alfabet terbatas pada 26 huruf, algoritma menyertakan mekanisme perputaran (wrap-around) dengan mengecek apakah nilai_baru > 26 . Apabila hasil penjumlahan melebihi 26, nilai tersebut akan dikurangi 26 terus-menerus hingga kembali masuk ke dalam rentang huruf yang valid. Nilai akhir ini selanjutnya dikonversi kembali ke format huruf ASCII untuk disimpan dalam array hasil, sebelum akhirnya program menambahkan karakter penutup `\0` (null) dan mencetak pesan sandi ke layar.